

# Campaign Management Platform

06 June 2026 14:56

You are a Principal Software Architect, Senior Product Manager, Staff Full-Stack Engineer, UI/UX Architect, Security Architect, DevOps Engineer, QA Lead, and Technical Writer.

Your objective is to DESIGN, DEVELOP, and DELIVER a COMPLETE production-quality self-hosted Campaign Management Platform.

Do not provide high-level suggestions or generic explanations.

Act as if you are building this software for a paying enterprise customer and must deliver implementation-ready artifacts, source code structure, database design, APIs, UI specifications, workflows, deployment setup, and development roadmap.

The platform must be fully implementable by a development team with minimal additional design decisions.

## PRODUCT OVERVIEW

Build a self-hosted Omnichannel Campaign Management Platform supporting:

- Email Campaigns
- SMS Campaigns
- Push Notification Campaigns

The platform must support:

- Campaign creation
- Campaign scheduling
- Campaign execution
- Audience management
- Segmentation
- Templates
- Analytics
- Reporting
- Approval workflows
- Notification provider integrations

The solution must be lightweight, modular, maintainable, secure, and deployment-ready.

## TECHNOLOGY STACK

Use ONLY the following technologies:

### Frontend

- React
- TypeScript
- Vite
- Material UI (MUI)
- React Router
- TanStack Query
- Zustand
- Axios
- Recharts

### Backend

- Python 3.12+
- FastAPI
- SQLAlchemy

- Alembic
- Pydantic
- Uvicorn

## Database

- SQLite

## Storage

Use local JSON files for:

- Configuration
- Sample Data
- Metadata
- Provider Configurations
- Lookup Values

## Deployment

Everything must run locally .

## MODULES TO BUILD

Generate complete implementation design for:

### Authentication & Authorization

Features:

- Login
- Logout
- JWT Authentication
- Refresh Tokens
- Password Reset
- User Profile

Roles:

- Admin
- Marketer
- Viewer

Generate:

- RBAC matrix
- API security model
- Authentication flows

## Campaign Management

Support:

### Campaign Types

- One-Time Campaign
- Recurring Campaign
- Basic Drip Campaign
- Multi-Channel Campaign

### Campaign Lifecycle

- Draft
- Pending Approval
- Approved
- Scheduled
- Sending
- Completed

- Failed
- Paused
- Cancelled
- Archived

## Features

- Create Campaign
- Edit Campaign
- Duplicate Campaign
- Pause Campaign
- Resume Campaign
- Archive Campaign
- Delete Campaign
- Approval Workflow
- Campaign Calendar View

Generate:

- Functional Requirements
- UI Specifications
- APIs
- Database Design
- Workflow Diagrams
- State Transition Diagrams

## Template Management

Centralized template library.

### Email Templates

Support:

- HTML Content
- Subject
- Preheader
- Personalization Variables

### SMS Templates

Support:

- Text Content
- Character Counter
- Link Tracking

### Push Templates

Support:

- Title
- Body
- Image URL
- Deep Link
- Buttons

Features:

- Clone
- Search
- Categorize
- Versioning
- Preview
- Archive

Generate:

- Database Design

- APIs
- UI Design
- Validation Rules

## **Audience Management**

Support:

### **Contact Management**

- CSV Upload
- Bulk Import
- Manual Entry

### **Contact Data**

- Standard Fields
- Custom Fields
- Tags

### **Segmentation**

Support:

- Filter Builder
- Dynamic Segments
- Saved Segments

Generate:

- Segment Engine Design
- APIs
- Database Design
- UI Design

## **Consent Management**

Support:

### **Email**

- Subscription Status
- Unsubscribe Handling

### **SMS**

- STOP
- START
- HELP

### **Push**

- Device Consent

Compliance Features:

- GDPR Ready
- TCPA Ready
- CAN-SPAM Ready

Generate:

- Database Design
- Workflow Design
- APIs

## **Provider Integration Framework**

Design a pluggable provider architecture.

### **Email Providers**

- SMTP
- SendGrid

## **SMS Providers**

- Twilio

## **Push Providers**

- Firebase Cloud Messaging
- OneSignal

Requirements:

- Adapter Pattern
- Provider Factory
- Health Checks
- Retry Handling
- Provider Switching

Generate:

- Interface Definitions
- UML Diagrams
- Sequence Diagrams
- Example Implementations

## **Campaign Execution Engine**

Design lightweight background processing.

Use:

- FastAPI Background Tasks
- AsyncIO

Support:

## **Scheduling**

- Immediate Send
- Scheduled Send
- Timezone Support

## **Processing**

- Batch Processing
- Retry Logic
- Rate Limiting

## **Failure Handling**

- Retry Policy
- Dead Letter Handling
- Error Logging

Generate:

- Architecture Design
- Execution Flows
- Sequence Diagrams

## **Event Tracking Framework**

Track:

## **Email**

- Sent
- Delivered
- Opened
- Clicked

- Bounced
- Complaints
- Unsubscribed

## **SMS**

- Sent
- Delivered
- Failed
- Replied

## **Push**

- Sent
- Delivered
- Opened
- Action Clicked

Generate:

- Event Schema
- Tracking APIs
- Analytics Storage Design

## **Analytics Dashboard**

Provide:

### **Campaign Analytics**

- Sent
- Delivered
- Opened
- Clicked
- Converted

### **Channel Analytics**

Email:

- Open Rate
- Click Rate
- Bounce Rate

SMS:

- Delivery Rate
- Failure Rate
- Reply Rate

Push:

- Open Rate
- Action Rate

Generate:

- Dashboard Design
- Aggregation Logic
- Database Design
- APIs

## **Reporting Module**

Support:

- CSV Export
- Excel Export
- PDF Export

Scheduled Reports:

- Daily
- Weekly
- Monthly

Generate:

- Reporting Architecture
- APIs
- Database Design

## Audit Logging

Track:

- User Actions
- Campaign Changes
- Provider Configuration Changes
- Login Activities

Generate:

- Audit Schema
- APIs
- UI Design

## DATABASE DESIGN

Use SQLite.

Generate:

### ER Diagram

### Complete Schema

Tables:

- users
- roles
- permissions
- user\_roles
- campaigns
- campaign\_steps
- templates
- contacts
- contact\_custom\_fields
- segments
- segment\_rules
- deliveries
- provider\_configs
- event\_logs
- analytics\_snapshots
- reports
- audit\_logs

For every table provide:

- Columns
- Data Types
- Constraints
- Indexes
- Relationships

## JSON STORAGE DESIGN

Use JSON files for:

config/

- app.json
- campaign.json
- analytics.json
- security.json

providers/

- smtp.json
- sendgrid.json
- twilio.json
- fcm.json
- onesignal.json

metadata/

- countries.json
- timezones.json
- campaign\_types.json

sample-data/

- contacts.json
- templates.json
- segments.json

Generate:

- Directory Structure
- JSON Schemas
- Sample Data Examples

## API DESIGN

Generate complete REST APIs.

For every endpoint provide:

- URL
- HTTP Method
- Request Schema
- Response Schema
- Validation Rules
- Error Responses
- Authorization Requirements

Modules:

- Authentication
- Users
- Roles
- Campaigns
- Templates
- Contacts
- Segments
- Providers
- Analytics
- Reports
- Audit Logs

Provide OpenAPI specifications.

## UI/UX DESIGN

Generate complete UI specifications.

Pages:



- Login
- Dashboard
- Campaign List
- Campaign Builder
- Campaign Details
- Template Library
- Contact Management
- Segment Builder
- Analytics
- Reports
- User Management
- Provider Configuration
- Audit Logs

For each page provide:

- Layout
- Components
- Actions
- Validation Rules
- Empty States
- Error States

Generate wireframes using Mermaid diagrams.

## SECURITY DESIGN

Implement:

- JWT Authentication
- Refresh Tokens
- Password Hashing (bcrypt)
- Input Validation
- SQL Injection Prevention
- XSS Protection
- Rate Limiting
- Secure Headers
- Audit Logging

Generate complete security architecture.

## TESTING STRATEGY

Generate:

- Unit Tests
- Integration Tests
- API Tests
- UI Tests
- Security Tests
- UAT Scenarios

## NON-FUNCTIONAL REQUIREMENTS

Target:

- 1,000 Concurrent Users
- 1,000,000 Contacts
- 500,000 Messages Per Day
- <2 Second UI Response Time
- Mobile Responsive UI

- 99.5% Availability

## FINAL DELIVERABLES

Produce the response in the following order:

1. Product Vision
2. Functional Requirements
3. High-Level Architecture
4. Low-Level Architecture
5. Database Design
6. JSON Storage Design
7. Backend Design
8. Frontend Design
9. API Specifications
10. Provider Integration Design
11. Campaign Execution Design
12. Analytics Design
13. Security Design
14. UI/UX Specifications
15. Deployment Design
16. Testing Strategy
17. Implementation Roadmap
18. Release Plan
19. Risks & Mitigations

Most importantly:

Generate the solution as if you are preparing it for actual implementation. Include source code structure, database schemas, API contracts, Mermaid diagrams, pseudocode where necessary, and enough technical detail that a development team can start building immediately without further architectural decisions.